URPEKARI PROIEKTUA & GALERNA UAS

TAP – Tiny Address Protocol

Started: 2025/07

Last revised: 2026/04
Allowing for reactive channel hopping
Link-Layer agnosticity

Address Header

Start Byte	Type	Field name	Description
0	uint16_t	SOF_WORD	Start of Frame indicator
2	uint8_t	TARGET_ID	Identifier for the receiving device.
3	uint8_t	SOURCE_ID	Identifier for the transmitting device.
4	uint8_t	MESSAGE_LENGTH	Payload length in bytes.
5	uint8_t	MESSAGE_TYPE	Message type indicator
6	uint16_t	COBS	

In 32 bit row format:

SOF_WORD		TARGET_ID	SOURCE_ID
MESSAGE_LENGTH	MESSAGE_TYPE	COBS	

Message trailer

We calculate the CRC of the header and the payload, but not the trailer.

Start Byte	Type	Field name	Description
m_len	uint16_t	CRC_16	Cyclic Redundancy Check
m_len + 2	uint16_t	SOF_WORD	End of Frame indicator

In 32 bit row format:

CRC_16	SOF_WORD

Message types

A number of message types have been reserved and used:

Value/Range	Name	Description
0	ACK/NACK	Positive or negative acknowledgement.
1	DIRECT_COMMAND	Traditional, direct RC control data.
2	INDIRECT_COMMAND	Waypoints.
3-15	Reserved	
16	TELEMETRY	Position and orientation information.
17-31	Reserved	
32-253	Available for mission-specific use-cases.	
254	NEGOTIATE_DATALINK	Datalink channel change command.
255	TELEMETRY_DATALINK	Datalink information.

ACK/NACK – 000

Start Byte	Type	Field name	Description
8	uint8_t	ACK_TYPE	Type of ACK message
9	*uint24_t*	TOKEN	Token for specific ACK uses

uint24_t is not a real type. Use a 3-length array of uint8_t instead.

The following ACK types are defined:

Value/Range	Name	Description
0	ACK	Positive acknowledgement.
1	NACK	Negative acknowledgement.
2-31	Reserved	
32-255	Available for mission-specific use-cases.	

DIRECT_COMMAND – 001

A direct command message carries two-byte groups. These can be interpreted as ranges for PWM signals (e.g.: the arduino map() function) or used for raw value transmission.

Start byte	Type	Field name	Description
8 + (2·n)	uint16_t	DATA_n	n^{th} pair of bytes

It is recommended to **reserve the first pair of bytes for up to 16 boolean values**, to operate lights, software-arming of motors and others.

Due to the Radiohead Library-established limit, a maximum of 251 bytes can be transmitted per packet. 123 signals can (in theory) be transmitted with this message type..

That limit is a lot higher than any real use-case is expected to reach, if more data needs to be transmitted, it should be split across multiple packets and the message types 2-15 should be used.

Urpekari implementation of Direct_Command

We have opted for a 16-bit 8-channel direct command payload. This should provide us with enough controls for anything on board any ROV, USV or UAS planned at this time. Further controls will be handled with additional message types.

Start Byte	Type	Field name	Description
8 + 0	**uint16_t**	**BOOLS**	**16 bit boolean-array**
8 + 2	uint16_t	Reserved	Reserved
8 + 4	uint16_t	Channel_0	Gradual control channel
8 + 6	uint16_t	Channel_1	Gradual control channel
...	...	...	...
8 + 18	uint16_t	Channel_7	Gradual control channel

In 32 bit row format:

BOOLS	Reserved
Channel_0	Channel_1
Channel_2	Channel_3
Channel_4	Channel_5
Channel_6	Channel_7

INDIRECT_COMMAND - 002

An indirect command carries waypoint information: GPS coordinates, altitude/depth information, and a radius indicating the required precission to “hit” the waypoint.

Start byte	**Type**	**Field name**	**Description *[unit]***
8 + 0	**uint16_t**	**BOOLS**	**16 bit boolean-array**
8 + 2	uint16_t	Reserved	Reserved
8 + 4	float	GPS_LAT	Waypoint GPS Latitude. [DEC]
8 + 8	float	GPS_LON	Waypoint GPS Longitude. [DEC]
8 + 12	uint16_t	ALT-DEP	Waypoint altitude or depth. [*cm*]
8 + 14	uint16_t	RADIUS	Waypoint precission radius. [*m*]

In 32 bit row format:

BOOLS	Reserved
GPS_LAT	
GPS_LON	
ALT-DEP	RADIUS

Boolean arrays for DIRECT_COMMAND and INDIRECT_COMMAND

While only INDIRECT_COMMAND messages require the 2-Byte boolean array at the start of the payload, it is recommended to do so in DIRECT_COMMAND messages.

This example shows a simple and practical way to use them to aid the control of an autonomous vehicle. What is described in the following section is what is implemented in the flight controller for the Galerna UAS:

Bit	**Name**	**Description when true (1)**
0	ARM	Allow for software arming of the thrust systems.
1	AUTO	Fly autonomously folowing waypoints. (This is overriden by DIRECT_COMMAND messages, even when they have the AUTO bit enabled.)
2	STAB	Partial autopilot, the UAS will attempt level navigation and filter inputs.
3	NAVLIGHT	Turn on navigation lights.
4	STROBE	Turn on strobe lights.
5	LAND	Extend landing gear or other physical supports. (If applicable.)
6	*N/A*	*Free so far*
7	*N/A*	*Free so far*
8	*N/A*	*Free so far*
9	*N/A*	*Free so far*
10	*N/A*	*Free so far*
11	COMLOSSBY	Bypass mid-flight communications loss detection and reaction. (Use with caution.)
12	LOITER	Loiter next waypoint, requires an INDIRECT_COMMAND. (If off, any waypoint will be treated as a fly-through waypoint.)
13	RTH	Return to home (First waypoint) directly. (Home may also be specified as the first GPS point registered.)
14	CHECKSUM	Checksum
15		

TELEMETRY - 016

A basic telemetry message carries information regarding the vehicle's current position.

Start byte	Type	Field name	Description *[unit]*
8 + 0	float	GPS_LAT	Current GPS Latitude. [DEC]
8 + 4	float	GPS_LON	Current GPS Longitude. [DEC]
8 + 8	uint16_t	ALT-DEP	Current altitude [m] or depth. [*cm*]
8 + 10	int16_t	HEADING	Heading. [º/10] (24.5º → 245 in this variable)
8 + 12	float	ROLL	Roll angle. [º]
8 + 16	float	PITCH	Pitch angle. [º]

In 32 bit row format:

GPS_LAT	
GPS_LON	
ALT-DEP	HEADING
ROLL	
PITCH	

NEGOTIATE_DATALINK- 254

A datalink control message, exclusively used by the radio ingest device. A channel hop is used when a channel that is in use is of low quality.

The information provided by the DATALINK_TELEMETRY is used to determine the quality of the channel in use. A channel hop may also be purposefully triggered by the user.

In the current version, channel options must be set up before code compilation.

TELEMETRY_DATALINK – 255

A datalink telemetry message, exclusively used by the radio ingest device. It's main purpose is to keep both sides of the conversation informed of datalink quality.

Start byte	Type	Field name	Description *[unit]*
8 + 0	int16_t	RSSI	Received Signal Strength Indicator. [dB]
8 + 2	uint16_t	SNR	Signal to Noise Ratio [dB]
8 + 4	uint16_t	RTT	Return trip time [ms]
8 + 6	uint16_t	SENT_PKTS	Amount of messages sent since the previous TELEMETRY_DATALINK message
8 + 8	uint16_t	DELTA_T	Time elapsed since the previous TELEMETRY_DATALINK message [ms]
8 + 10	uint16_t	Reserved	Reserved

Specific feature implementations

Message serializer

The TAP serializer works by receiving an incomplete header and a complete payload. The following table highlights the header fields the serializer requires:

SOF_WORD		TARGET_ID	SOURCE_ID
MESSAGE_LENGTH	MESSAGE_TYPE	COBS	

The raw payload will consist of arbitrary data, identified for the upper layers by the MESSAGE_TYPE field on the receiving end, and be of MESSAGE_LENGTH length.

Big endian byte orders have been used for all fields defined in this document.

COBS

Consistent Overhead Byte Stuffing (COBS) is a mechanism that protects the Start of Frame (SOF) word from appearing in illegal positions. It is necessary for un-managed links such as UART and bit-banged radio links, where the start and end of a frame are up for us to manage.

COBS as implemented by TAP uses a 2-byte field in every Address Header. The serializer and deserializer must handle COBS in a completely transparent way. It must be the last step taken before transmission and the first step taken upon reception. COBS protects header, payload and trailer from illegal SOF word appearances.

Let's take the following example, with the header highlighted in yellow and the trailer highlighted in green.

SOF_WORD		TARGET_ID	SOURCE_ID
MESSAGE_LENGTH	MESSAGE_TYPE	COBS	
DATA	DATA	DATA	DATA
DATA	DATA	DATA	DATA
DATA	DATA	DATA	DATA
DATA	DATA	DATA	DATA
DATA	DATA	DATA	DATA
CRC_16		SOF_WORD	

In a case with no illegal SOF word placement, the COBS field will simply be set to 0x0000 and the message will be transmitted with no more complication.

However, were the SOF word to show up at any point in the message, the COBS field in the header will instead point to the first byte of the illegal appearance of the SOF word. The illegal appearance of the SOF word will then be replaced by either

0x0000, if it is the last illegal appearance of the SOF word, or the pointer to the following appearance of an illegal SOF word.

Following along with the example, given this message:

SOF_WORD		TARGET_ID	SOURCE_ID
MESSAGE_LENGTH	MESSAGE_TYPE	COBS	
DATA	DATA	DATA	DATA
DATA	ILLEGAL SOF_WORD		DATA
DATA	DATA	DATA	DATA
DATA	DATA	ILLEGAL SOF_WORD	
DATA	DATA	DATA	DATA
CRC_16		SOF_WORD	

The COBS algorithm must replace the mentioned fields like so:

SOF_WORD		TARGET_ID	SOURCE_ID
MESSAGE_LENGTH	MESSAGE_TYPE	13	
DATA	DATA	DATA	DATA
DATA	22		DATA
DATA	DATA	DATA	DATA
DATA	DATA	0	
DATA	DATA	DATA	DATA
CRC_16		SOF_WORD	